

Deploy Insurance App to Kubernetes

Course Name: DevOps Foundation

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Sarthak Teredesai	EN22CS301879
2	Shrasti Gupta	EN22CS301925
3	Shruti Sharma	EN22CS301940
4	Sanidhya Singh Chouhan	EN22CS301864
5	Saumya Kumari	EN22CS301886

Group Name: 05D8

Project Number: DO-40

Industry Mentor Name: Ms. Aashruti Shah

University Mentor Name: Prof. Avnesh Kumar Joshi

Academic Year: 2025-26

Problem Statement & Objectives

1. Problem Statement

In the modern insurance industry, users are often required to analyze and compare multiple insurance policies before making a decision. However, most insurance documents are complex, lengthy, and filled with technical jargon, making them difficult for the average user to understand. This creates a significant gap between available information and user comprehension.

Traditional systems provide static information about policies but lack intelligent capabilities to interpret, simplify, and personalize the content. Users must manually read through large volumes of data, which is time-consuming and prone to misinterpretation. Additionally, there is no automated mechanism to compare multiple policies effectively and generate meaningful insights such as recommendations or suitability analysis.

Another major challenge lies in scalability and system deployment. Many existing solutions are either standalone applications or lack proper cloud-native architecture, making them inefficient in handling multiple users or scaling dynamically based on demand. These systems also suffer from dependency issues, lack of fault tolerance, and limited availability.

With the advancement of Artificial Intelligence, particularly Large Language Models (LLMs), there is an opportunity to build intelligent systems capable of understanding and processing human language. However, integrating such AI models into a scalable, production-ready system remains a challenge. Most implementations either focus only on AI capabilities or only on deployment, but not both together.

Furthermore, there is a lack of systems that utilize a multi-agent architecture, where different AI agents perform specialized tasks such as data retrieval, explanation, and comparison. Without such modular design, systems become rigid, less efficient, and harder to maintain.

Therefore, the core problem addressed in this project is:

To design and develop a scalable, AI-powered system that can intelligently analyze, simplify, and compare insurance policies, while being deployed using modern DevOps practices such as containerization and orchestration.

This project aims to bridge the gap between complex insurance data and user-friendly insights by combining Artificial Intelligence with cloud-native technologies like Docker and Kubernetes. The goal is to create a system that is not only intelligent but also scalable, reliable, and production-ready.

2. Project Objectives

The primary objective of this project is to design and develop an intelligent, scalable, and user-friendly system that simplifies the process of understanding and comparing insurance policies using Artificial Intelligence and modern cloud technologies. The system aims to bridge the gap between complex insurance data and user comprehension through automation and intelligent processing.

1. To Develop an AI-Powered Insurance Analysis System

The project aims to build a system that leverages advanced AI techniques, particularly Large Language Models (LLMs), to analyze insurance policies. The system should be capable of interpreting complex policy data and transforming it into structured, meaningful information that users can easily understand.

2. To Implement a Multi-Agent Architecture

A key objective is to design a modular system using multiple AI agents, each responsible for a specific task:

Researcher Agent for retrieving policy information

Explainer Agent for simplifying technical content

Comparator Agent for analyzing and comparing multiple policies

This separation of responsibilities ensures better scalability, maintainability, and efficiency.

3. To Simplify Complex Insurance Information

The system should convert technical and legal policy language into simple, human-readable explanations. It should also provide insights such as:

- Who should buy a policy
- Who should avoid it
- Key takeaways

This improves accessibility for non-technical users.

4. To Enable Intelligent Policy Comparison

Another objective is to allow users to compare multiple insurance policies side-by-side. The system should generate structured comparisons and provide recommendations based on user needs, enabling better decision-making.

5. To Build a Scalable and Robust System Using DevOps Practices

The project aims to implement modern DevOps practices by:

- Containerizing the application using Docker
- Deploying it using Kubernetes
- Ensuring scalability, high availability, and fault tolerance

This makes the system production-ready and capable of handling multiple users efficiently.

6. To Develop a User-Friendly Web Interface

The system should include an intuitive frontend (Streamlit-based) where users can easily:

- Select policies
- Request analysis
- View results in a clear format

This ensures ease of use and better user experience.

7. To Ensure System Reliability and Performance

The system should be designed to handle failures gracefully, maintain consistent performance, and provide reliable outputs. Kubernetes orchestration helps achieve:

- Load balancing
- Auto-restart of failed components
- Zero-downtime deployment

8. To Integrate AI with Cloud-Native Architecture

A major objective is to combine AI capabilities with cloud-native deployment strategies. This ensures that the system is not just intelligent but also scalable, maintainable, and aligned with modern industry practices.

3. Scope of the Project

The scope of this project defines the boundaries, functionalities, and limitations of the proposed Multi-Agent Insurance System. It outlines what the system is designed to achieve, the technologies involved, and the extent to which it operates within the defined environment.

1. Functional Scope

The system is designed to perform three primary functions:

- a) Insurance Policy Research

The system allows users to select a specific insurance policy and retrieve detailed information about it. The AI-powered Researcher Agent processes the query and gathers structured data related to the policy.

b) Policy Explanation

The Explainer Agent simplifies complex insurance terminology into easy-to-understand language. It provides:

- A simplified explanation of the policy
- Recommendations on who should buy the policy
- Identification of users who should avoid it
- Key takeaways for quick understanding

c) Policy Comparison

The system enables users to compare two insurance policies side-by-side. The Comparator Agent analyzes both policies and generates:

- Structured comparisons
- Advantages and disadvantages
- Final recommendation based on analysis

2. Technical Scope

The project is implemented using a modern, cloud-native architecture:

Frontend: Streamlit-based web interface for user interaction

Backend: FastAPI for handling API requests and orchestration

AI Layer: CrewAI agents integrated with Gemini LLM

Deployment: Docker for containerization and Kubernetes for orchestration

The system is deployed on a Minikube Kubernetes cluster, making it suitable for local development and testing environments.

3. Deployment Scope

The project includes full DevOps implementation:

- Containerization using Docker
- Image storage using DockerHub
- Deployment using Kubernetes (Pods, Services, Deployments)
- Exposure via NodePort services

The system supports:

- Scalability through multiple pod replicas
- Fault tolerance via automatic restarts
- Zero-downtime updates using rolling deployments

4. Data Scope

The system operates on predefined insurance policies and does not rely on a traditional database. Data is processed dynamically during runtime using AI models. Key characteristics include:

- No persistent storage of user data
- In-memory processing of requests and responses
- API-based communication between components

5. User Scope

The system is intended for:

- General users seeking insurance understanding
- Students and learners exploring AI + DevOps integration
- Developers and evaluators analyzing system design

The interface is designed to be simple and accessible, requiring no technical expertise.

6. Limitations of the Scope

Despite its capabilities, the system has certain limitations:

- It does not fetch real-time policy data from external APIs
- The accuracy of results depends on the AI model (LLM responses)
- It is deployed on Minikube (local cluster), not a full cloud production environment
- No user authentication or personalization features are implemented
- Limited to predefined policy datasets

7. Future Expandability within Scope

Although the current implementation is limited, the system is designed in a modular way, allowing future enhancements such as:

- Integration with real insurance APIs
- Deployment on cloud platforms (AWS, GCP, Azure)
- Addition of more AI agents and functionalities

Proposed Solution

1. Key Features

The proposed solution is a **Multi-Agent AI-powered Insurance System** that intelligently processes, simplifies, and compares insurance policies using Artificial Intelligence and cloud-native technologies.

It addresses the limitations of traditional static systems by introducing:

- Automation
 - Scalability
 - Intelligent decision-making
-

2. Overview of the Solution

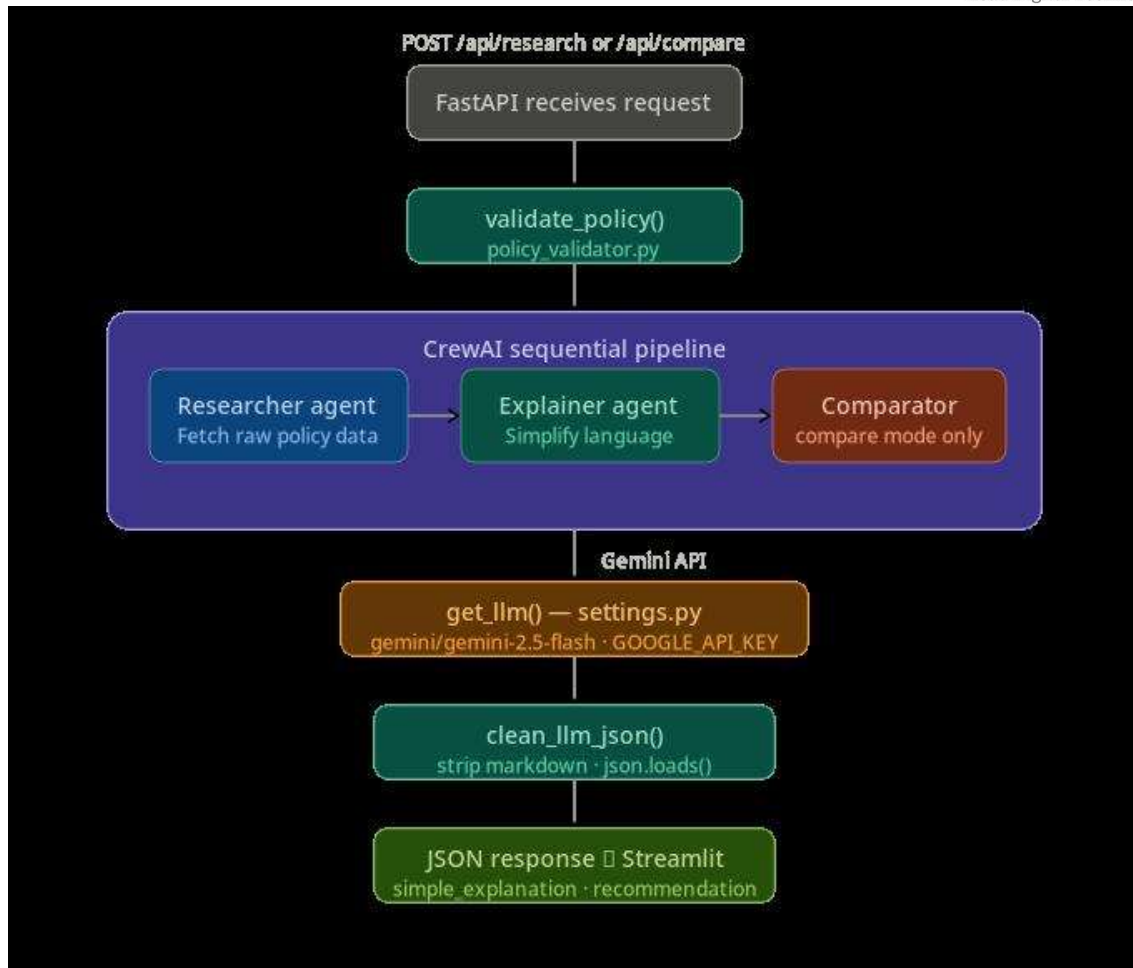
The system follows a **three-tier architecture**:

- **Presentation Layer (Frontend)**
- **Application Layer (Backend)**
- **AI/Intelligence Layer (Agents + LLM)**

Users interact through a web interface. Their requests are processed by the backend and handled by specialized AI agents, delivering structured and user-friendly outputs.

3. Multi-Agent Architecture

The system is built on a modular **multi-agent design**, where each agent performs a specific role:



a) Researcher Agent

- Retrieves and processes raw insurance policy data
- Uses LLM to extract structured insights
- Acts as the data provider

b) Explainer Agent

- Simplifies complex policy information
- Generates:
 - Easy explanations
 - “Who should buy” recommendations
 - “Who should avoid” suggestions
 - Key takeaways

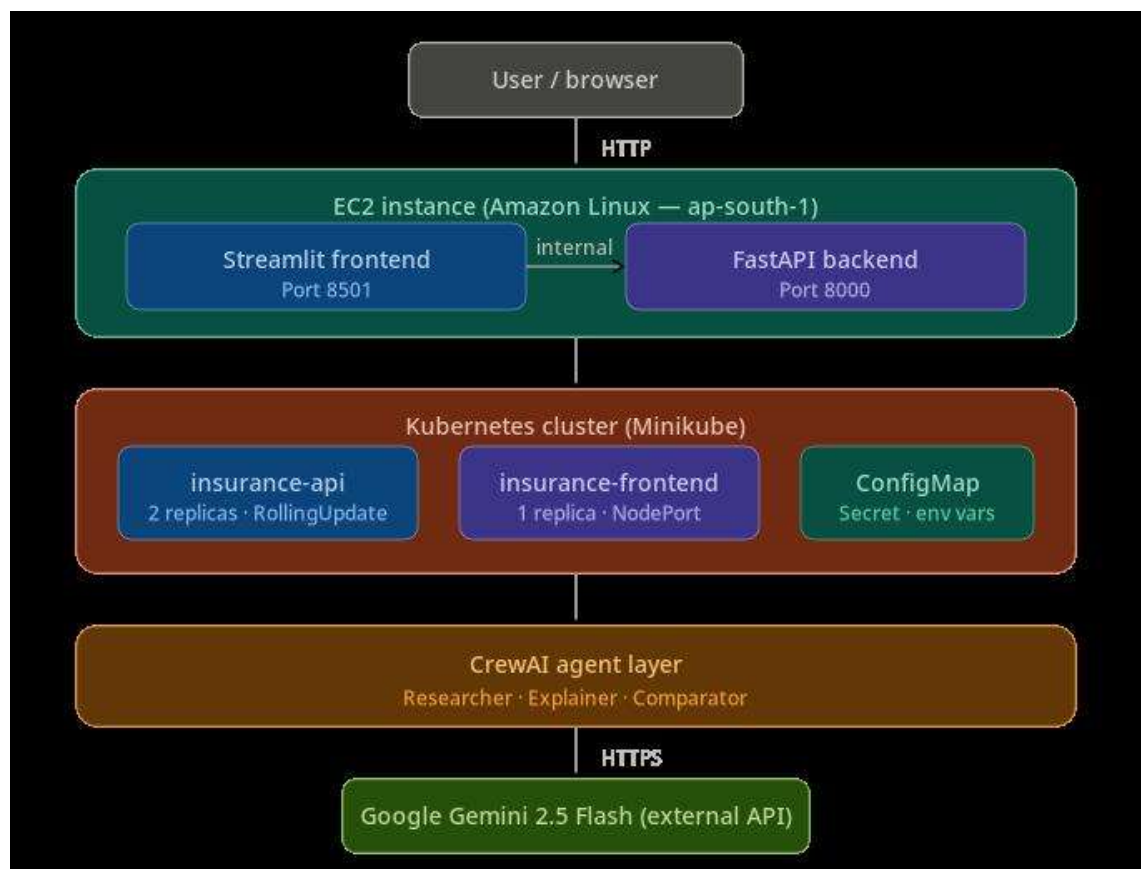
c) Comparator Agent

- Compares multiple insurance policies
- Generates structured comparisons
- Provides recommendations for decision-making

- Improved efficiency
- Better scalability
- Easier maintenance

4. System Workflow

1. User selects a policy or comparison request via frontend
2. Request is sent to backend using REST API
3. Backend validates input and triggers AI agents
4. Agents interact with LLM (Gemini)
5. Structured output is generated
6. Results are displayed on frontend



5. Technology Stack

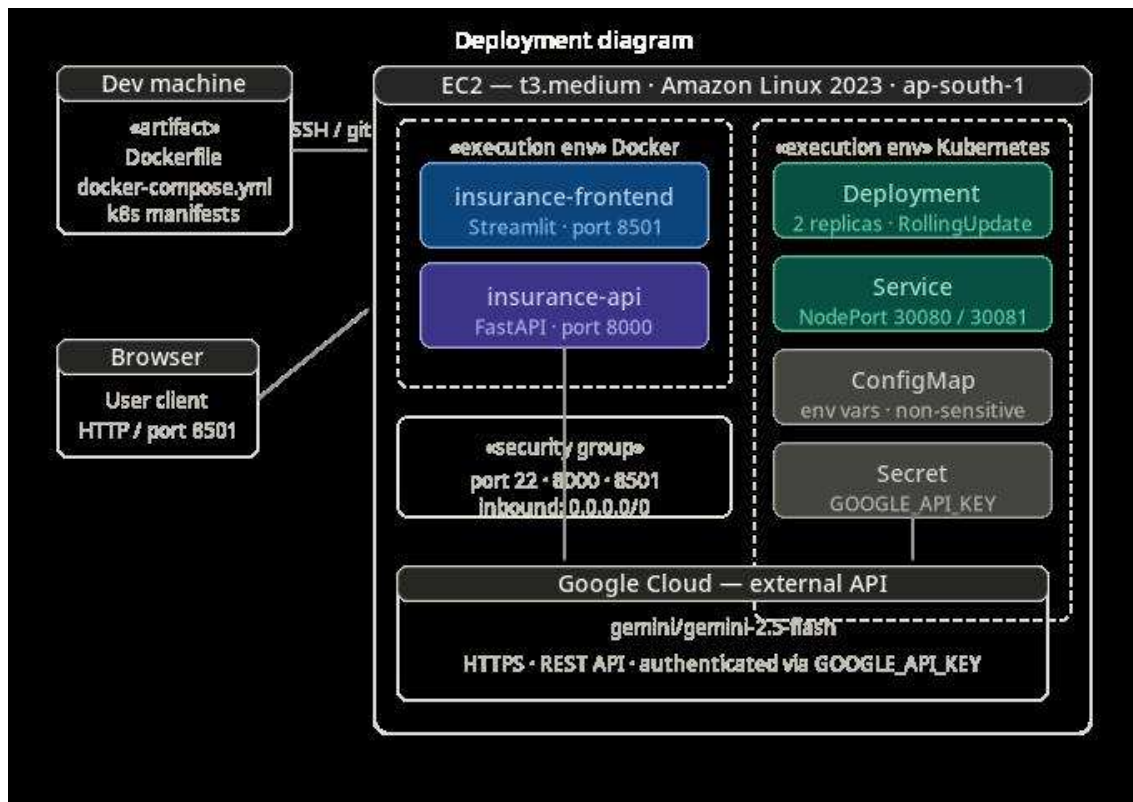
Component	Technology Used
Frontend	Streamlit

Component	Technology Used
Backend	FastAPI
AI Framework	CrewAI
LLM	Google Gemini
Containerization	Docker
Orchestration	Kubernetes (Minikube)

6. Deployment Strategy

The system follows **DevOps best practices**:

- Docker for containerization
- DockerHub for image storage
- Kubernetes for orchestration



Kubernetes Components

- Pods – Running containers
- Deployments – Replica management
- Services – External access

- **Scalability** – Handles multiple requests
 - **Fault Tolerance** – Auto-restart of failed containers
 - **Zero Downtime** – Rolling updates
-

7. Key Features of the Solution

- AI-based policy analysis
 - Simplified explanations
 - Intelligent policy comparison
 - Cloud-native scalable deployment
 - Modular multi-agent design
 - User-friendly interface
-

8. Advantages

- Reduces complexity for users
 - Automates decision-making
 - Efficient handling of multiple requests
 - Easy scalability and maintenance
 - Combines AI with real-world deployment
-

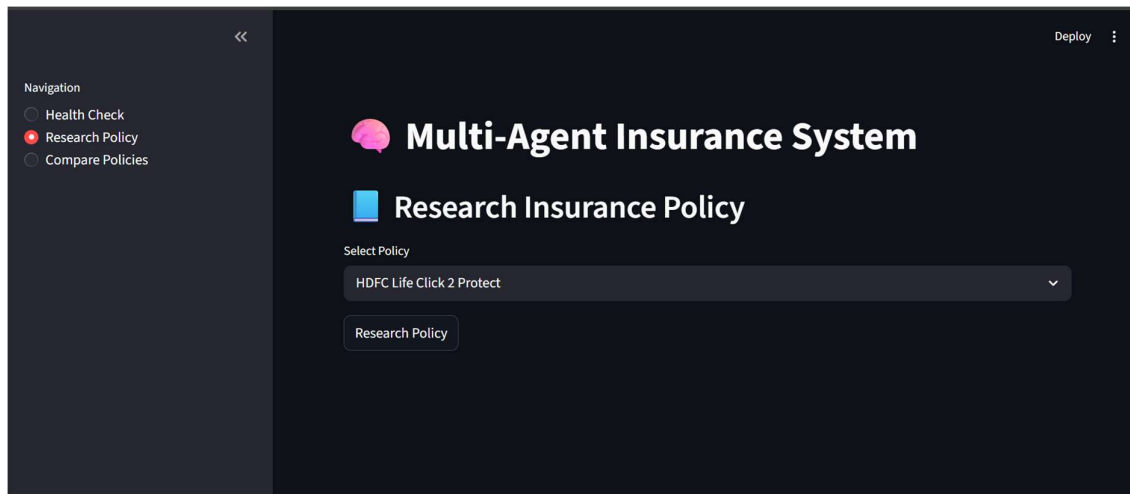
9. Innovation

- Multi-agent AI architecture
 - Integration of LLM + DevOps (Docker + Kubernetes)
 - Real-time structured responses
 - Stateless and scalable system design
-

Results & Output

1. Overview

The system was successfully implemented and deployed in a **containerized Kubernetes environment**, demonstrating its ability to process queries and generate meaningful outputs using AI agents.



2. Functional Results

a) Policy Research Output

- Generates detailed policy insights
- Uses Researcher Agent for structured extraction
- Outputs data in JSON format

```
Agent Started

Agent: Insurance Policy Research Specialist

Task:
  Research the insurance policy: HDFC Life Click 2 Protect

  You MUST return output strictly in JSON format.

  JSON structure:
  {
    "policy_name": "HDFC Life Click 2 Protect",
    "summary": "Brief 4-5 line summary",
    "coverage": "Coverage details",
    "premium": "Premium details",
    "pros": ["point1", "point2"],
    "cons": ["point1", "point2"],
    "best_for": "Type of customer this is ideal for"
  }

  Do NOT return explanation.
  Do NOT return markdown.
  Return ONLY valid JSON.
```

b) Policy Explanation Output

The Explainer Agent provides:

- Simplified explanations
- “Who should buy” recommendations
- “Who should avoid” suggestions
- Key takeaways

c) Policy Comparison Output

The Comparator Agent generates:

- Side-by-side comparisons
- Pros and cons
- Final recommendations

Policy Overview

Simple Explanation

HDFC Life Click 2 Protect is a flexible term insurance plan designed to financially protect your family if something happens to you. It offers various options, including pure life cover, critical illness benefits, increasing coverage, or even getting your premiums back, allowing you to customize protection at affordable rates.

Who Should Buy It

- Individuals seeking comprehensive and flexible financial protection for their families.
- Young professionals and families with dependents (spouse, children, elderly parents).
- Those with significant financial liabilities like home loans or other debts.
- People looking for customizable coverage, including critical illness benefits or the option to increase cover at life stages.
- Individuals who want the option to get their premiums back if they survive the policy term (Return of Premium option).

3. System Performance

- **Response Time:** 30–120 seconds (based on complexity)
 - **Scalability:** Multiple Kubernetes pods handle concurrent users
 - **Availability:** Auto-recovery from failures
-

4. Deployment Results

- Docker ensured consistent environments
 - Kubernetes handled orchestration
 - NodePort enabled external access
 - Rolling updates ensured zero downtime
-

5. Output Representation

The system uses a **Streamlit interface** where users can:

- Select policies
 - View structured explanations
 - Compare policies visually
-

6. Sample Output (Conceptual)

Policy Explanation

- Summary of policy
- Who should buy
- Who should avoid
- Key takeaways

Policy Comparison

- Best policy recommendation
 - Feature-wise comparison table
-

7. Key Outcomes

- Successful integration of AI agents with backend
 - Accurate structured outputs using LLM
 - Smooth frontend-backend-agent interaction
 - Efficient cloud-native deployment
-

8. Limitations

- Response time depends on LLM processing
- Output may vary due to AI generation
- Limited to predefined policies

Final Summary

The system successfully demonstrates an **intelligent, scalable, and automated insurance solution**. It combines AI capabilities with modern DevOps practices, delivering both **functional efficiency and technical robustness**.

Conclusion

The Multi-Agent Insurance System successfully demonstrates the integration of Artificial Intelligence with modern cloud-native technologies to solve real-world problems in the insurance domain. The project addresses the challenge of understanding and comparing complex insurance policies by introducing an intelligent, automated, and user-friendly solution.

The system effectively utilizes a multi-agent architecture, where specialized AI agents perform tasks such as data retrieval, explanation, and comparison. This modular design enhances the efficiency, scalability, and maintainability of the system. By leveraging Large Language Models (LLMs), the system is able to generate structured and meaningful outputs, improving user comprehension and decision-making.

From a technical perspective, the implementation of Docker and Kubernetes ensures that the system is scalable, reliable, and production-ready. Features such as containerization, orchestration, fault tolerance, and zero-downtime deployment highlight the robustness of the solution.

The project also demonstrates a successful interaction between multiple layers—frontend (Streamlit), backend (FastAPI), and AI agents (CrewAI + Gemini)—resulting in a seamless end-to-end workflow. Users are able to interact with the system easily and receive clear, actionable insights.

However, the system is currently limited to predefined datasets and depends on AI-generated outputs, which may vary slightly in accuracy. Despite these limitations, the project provides a strong foundation for building advanced, real-world AI-driven applications.

In conclusion, this project not only fulfills its objectives but also showcases the potential of combining AI with DevOps practices to create scalable and intelligent systems. It serves as a

Future Scope & Enhancements

The current implementation of the Multi-Agent Insurance System provides a strong foundation for intelligent policy analysis; however, there are several opportunities to enhance its capabilities and extend its real-world applicability.

1. Integration with Real-Time Insurance Data

The system currently operates on predefined policy data. In the future, it can be integrated with:

- Insurance company APIs
- Real-time policy databases
- This would enable dynamic data retrieval, ensuring that users receive up-to-date and accurate information.

2. Deployment on Cloud Platforms

The system is currently deployed on a local Kubernetes cluster (Minikube). It can be extended to cloud platforms such as:

- Amazon Web Services (AWS)
- Google Cloud Platform (GCP)
- Microsoft Azure

This would improve scalability, accessibility, and real-world usability.

3. Advanced Personalization

Future versions can include:

- User profiles
- Personalized recommendations based on age, income, and preferences
- Adaptive AI responses

This would make the system more user-centric and decision-oriented.

4. Expansion of AI Capabilities

The system can be enhanced by:

- Adding more specialized agents (e.g., Risk Analyzer, Fraud Detector)
- Using more advanced or fine-tuned LLMs
- Improving accuracy and consistency of outputs

This would increase the intelligence and reliability of the system.

5. Improved User Interface and Experience

upgraded to:

- A full-stack web application (React + backend)
- Mobile application support
- Interactive dashboards and visualizations

This would enhance usability and user engagement.

6. Security and Authentication

Future improvements can include:

- User authentication and authorization
- Secure API access
- Data encryption and privacy controls

This is essential for real-world deployment, especially in sensitive domains like insurance.

7. Performance Optimization

Enhancements can be made to:

- Reduce AI response latency
- Implement caching mechanisms
- Optimize API response times

This would improve overall system efficiency.

8. Multi-Domain Expansion

The architecture can be extended beyond insurance to other domains such as:

- Banking and finance
- Healthcare
- E-commerce

This makes the system reusable and adaptable across industries.

Final Future Vision

The project has the potential to evolve into a fully production-grade AI platform that provides intelligent decision support at scale. By integrating real-time data, cloud infrastructure, and advanced AI models, the system can become a powerful tool for both users and organizations.